

Week-6

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Ans: Spark Architecture Theory

- **Driver:** Creates SparkSession, converts user code into jobs, schedules tasks and collects results.
- **Cluster Manager:** Allocates resources and manages worker nodes.
- **Executor:** Executes tasks, stores data in memory and returns results to Driver.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chainprocessing large datasets?

Ans: Spark does not execute transformations immediately. Instead, it creates a **Directed Acyclic Graph (DAG)** and optimizes the execution plan before performing actions. This reduces unnecessary computations and improves performance.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and `inferSchema` is enabled.

Ans: import pandas as pd

Create sample csv file

```
sample = pd.DataFrame({  
    "product_id":[101,102,103,104],  
    "category":["Electronics","Books","Electronics","Clothes"],  
    "price":[1500,500,3000,700]  
})
```

```
sample.to_csv("source.csv",index=False)
```

Read CSV using Spark

```
df_csv = spark.read \
```

```
.option("header",True) \

.option("inferSchema",True) \

.csv("source.csv")

df_csv.show()
```

product_id	category	price
101	Electronics	1500
102	Books	500
103	Electronics	3000
104	Clothes	700

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Ans: The difference is:

CSV	Parquet
Row-based storage	Columnar storage
Large file size	Compressed
Slower	Faster
No schema support	Schema supported

Performance:

Parquet performs better because Spark reads only required columns.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Ans: electronics = df.select(
 "product_id",

```

    "price"
).filter(

    df.category=="Electronics"

)

electronics.show()

```

```

+-----+
|product_id|price|
+-----+
|      101| 1500|
|      103| 3000|
+-----+

```

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the `price` column from a `String` to a `Double`.

Ans: from pyspark.sql.functions import col

Rename column

```

df = df.withColumnRenamed(

    "product_id",

    "new_name"

)

```

Cast datatype

```

df = df.withColumn(

    "price",

    col("price").cast("double")

)

```

```
df.printSchema()
```

```
root
 |-- new_name: long (nullable = true)
 |-- category: string (nullable = true)
 |-- price: double (nullable = true)
```

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Ans: Spark stores lineage information through DAG. If an executor fails, Spark recomputes only the lost partitions instead of the entire dataset, providing fault tolerance.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

Ans: orders = [

(1,"Completed",2000),

(2,"Pending",500),

(3,"Completed",1500),

(4,"Completed",400)

]

```
df_orders = spark.createDataFrame(
```

```
    orders,
```

```
    ["id","status","amount"]
```

```
)
```

```
result = df_orders.filter(
```

```
    (df_orders.status=="Completed") &
```

```
    (df_orders.amount>1000)
```

)

```
result.show()
```

id	status	amount
1	Completed	2000
3	Completed	1500

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Ans: Predicate Pushdown applies filters while reading Parquet files. This reduces disk I/O, memory usage and improves query performance because unnecessary data is never loaded.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

Ans: `df = df.withColumn(`

`"final_price",`

`col("price")*1.18`

)

```
df.show()
```

new_name	category	price	final_price
101	Electronics	1500.0	1770.0
102	Books	500.0	590.0
103	Electronics	3000.0	3540.0
104	Clothes	700.0	826.0

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Ans: Transformations are lazy operations whereas actions trigger execution.

Transformations

- filter()
- select()

Actions

- show()
- count()

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Ans: data = [

(1,"A"),

(2,"B"),

(None,"C")

]

df = spark.createDataFrame(

data,

["user_id","name"]

)

Save parquet

df.write.mode("overwrite").parquet("input_parquet")

Read parquet

df2 = spark.read.parquet(

```

    "input_parquet"
)

# Filter null values

df2.filter(

    df2.user_id.isNotNull()

).write.mode(

    "overwrite"

).csv(

    "output_csv"

)

print("Saved Successfully")

```

+-----+-----+	
user_id name	
+-----+-----+	
1 A	
2 B	
NULL C	
4 D	
+-----+-----+	
+-----+-----+	
user_id name	
+-----+-----+	
1 A	
2 B	
4 D	
+-----+-----+	

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Ans:

Client Mode

Driver runs on local machine

Used for development

Cluster Mode

Driver runs inside cluster

Used for production

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Ans: data = [

 ("North","Low"),

 ("South","High"),

 ("North","High"),

 ("East","Low")

]

df_region = spark.createDataFrame(

 data,

 ["region","priority"]

)

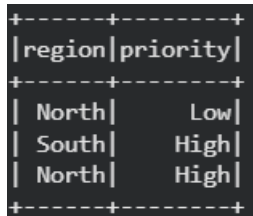
result = df_region.filter(

 (df_region.region=="North") |

 (df_region.priority=="High")

)


```
result.show()
```

A terminal window showing the output of a Spark SQL query. The output is a table with two columns: 'region' and 'priority'. The table contains three rows of data: North with Low priority, South with High priority, and North with High priority. The table is displayed with a header row and data rows, separated by dashed lines.

region	priority
North	Low
South	High
North	High

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Ans:

- `show(5)` retrieves only a few rows and is safe for large datasets.
- `collect()` loads the entire dataset into driver memory and may cause memory overflow on multi-terabyte datasets.

Performance Insights

1. Spark uses Lazy Evaluation for optimization.
2. DAG provides fault tolerance.
3. Parquet performs better than CSV.
4. Predicate Pushdown reduces I/O.
5. Wide transformations involve shuffle.
6. `show()` is safer than `collect()`.
7. Spark performs in-memory processing.